

ANTICITHERA

Hub de Inteligencia Científica

Manual Técnico

Colaborador	Rol Principal
Fernando Álvarez García	Backend, Infraestructura y Servicios
Cristopher Ponce Carrera	Frontend, Arquitectura y Orquestación

Ingeniería de Software 2 · 2026

Tabla de Contenidos

Tabla de Contenidos	2
1. Introducción	4
1.1 Colaboradores	4
1.2 Stack Tecnológico	4
2. Planteamiento del Problema	5
3. Requerimientos Funcionales	5
3.1 Módulo «La Librería»	5
3.2 Módulo «Motor de Fichas»	5
3.3 Módulo «Laboratorio»	6
3.4 Funcionalidades Generales	6
4. Arquitectura del Sistema	7
4.1 Diagrama de Capas	7
4.2 Backend: Estructura de Paquetes	7
4.3 Esquema de Base de Datos	7
4.4 Seguridad CORS	8
5. Diagramas de Casos de Uso	9
5.1 Actor: Usuario Investigador	9
5.2 Sistema	10
5.3 Flujo de Trabajo Típico	11
5.4 Diagrama de clases	11
6. Internacionalización (i18n)	12
6.1 Motor de i18n	12
6.2 Atributos HTML para Traducción	12
6.3 Estructura de los Archivos JSON	12
6.4 Cambio de Idioma en Tiempo Real	13
7. Integración con Google Cloud (Google Drive)	14
7.1 Configuración en Google Cloud Console	14
7.2 Implementación en el Frontend	14
7.3 Descarga de Archivos	14
8. Calidad y Seguridad del Código	15
8.1 SonarCloud — Análisis de Calidad	15
8.2 CodeQL — Análisis de Seguridad	16
8.3 Prácticas de Seguridad Implementadas	16

9. Referencia de la API REST	17
9.1 Autenticación — /api/auth	17
9.2 Archivos ZIP — /api/files	17
10. Guía de Despliegue	18
10.1 Requisitos del Sistema	18
10.2 Configuración del Backend	18
10.3 Configuración del Frontend	18
11. Glosario	19

1. Introducción

Anticithera es una plataforma web de investigación científica diseñada como un "Hub de Inteligencia Científica". Su propósito central es centralizar y optimizar el ciclo de trabajo académico: desde la gestión de documentos y la anotación bibliográfica hasta la ejecución interactiva de cuadernos Jupyter y la sincronización de respaldos en la nube.

El nombre del proyecto rinde homenaje al Mecanismo de Anticitera, considerado el primer ordenador analógico de la historia, hallado en un barco hundido en el mar Egeo y datado aproximadamente en el siglo I a.C. Al igual que aquel artefacto fue una herramienta de conocimiento que adelantó su tiempo, Anticithera aspira a ser un instrumento que potencie la capacidad investigativa de sus usuarios.

1.1 Colaboradores

Colaborador	Rol	Repositorio
Fernando Álvarez García	Backend · BD · Infraestructura	GitHub / F3rnando14
Cristopher Ponce Carrera	Frontend · Arquitectura · QA	GitHub / cristopher-ing-28

1.2 Stack Tecnológico

Capa	Tecnología	Versión / Notas
Frontend	HTML5 · Tailwind CSS · JavaScript ES2023	SPA monolítica
Backend	Jakarta EE 10 / JAX-RS	GlassFish 7
Persistencia	MariaDB + JPA (EclipseLink)	setup.sql incluido
Almacenamiento local	IndexedDB + LocalStorage	Funciona offline
Servicios externos	Google Drive API (Google Cloud)	OAuth 2.0
Seguridad backend	BCrypt + JWT (jjwt 0.11.5)	Tokens UUID
Análisis de calidad	SonarCloud	82.6 % de cobertura
Análisis de seguridad	CodeQL (GitHub Advanced Security)	Integrado en CI
CI/CD	Jenkins Pipeline	Build → Deploy → GH Pages

2. Planteamiento del Problema

En el ámbito de la investigación académica, los equipos de trabajo suelen enfrentar cuatro grandes obstáculos que reducen su productividad y cohesión:

- Dispersión de recursos: documentos en múltiples formatos (PDF, DOCX, .ipynb) almacenados en ubicaciones distintas, dificultando su recuperación y comparación.
- Ineficiencia en la anotación: procesos manuales y desorganizados para extraer fragmentos clave y generar fichas bibliográficas, con alto riesgo de errores de atribución.
- Falta de integración: herramientas de lectura, anotación y análisis que no se comunican entre sí, interrumpiendo el flujo de trabajo continuo.
- Dificultad en la exportación colaborativa: ausencia de un sistema unificado para compilar y respaldar el trabajo en formatos portables.

Anticithera aborda estos problemas con un entorno integrado que unifica lectura, anotación, análisis de notebooks y respaldo en la nube, todo desde una sola interfaz.

3. Requerimientos Funcionales

3.1 Módulo «La Librería»

ID	Descripción
RF1.1	Carga y organización de documentos en formatos PDF, DOCX y .ipynb en una librería local.
RF1.2	Visor de PDF integrado con capa de texto seleccionable (PDF.js).
RF1.3	Vinculación con directorios locales mediante File System Access API.
RF1.4	Importación de documentos desde Google Drive (OAuth 2.0).
RF1.5	Exportación de la librería como archivo ZIP con metadatos JSON.

3.2 Módulo «Motor de Fichas»

ID	Descripción
RF2.1	Selección de fragmentos de texto directamente desde el visor de documentos.
RF2.2	Generación automática de fichas con referencia de origen y número de página.
RF2.3	Campo de comentario y análisis personal por cada ficha.
RF2.4	Organización de fichas en colecciones personalizables con drag & drop.
RF2.5	Buscador de fichas por contenido y metadatos.

3.3 Módulo «Laboratorio»

ID	Descripción
RF3.1	Renderizado de cuadernos Jupyter (.ipynb) con notebook.js.
RF3.2	Ejecución interactiva de celdas de código en tiempo real.
RF3.3	Soporte de salidas: texto, imágenes, gráficos y ecuaciones LaTeX (MathJax).

3.4 Funcionalidades Generales

ID	Descripción
RF4.1	Buscador global entre documentos, fichas y proyectos.
RF4.2	Internacionalización (i18n): Español e Inglés mediante archivos JSON de traducción.
RF4.3	Modo oscuro (Dark Mode) con paleta Midnight / Ámbar.
RF4.4	Gestión de sesiones con autenticación JWT y registro de actividad.
RF4.5	Dashboard con métricas en tiempo real (documentos, fichas, sesión activa).
RF4.6	Creación de proyectos con tipos predefinidos: Artículo, Tesis, Tesina, Ensayo, Personalizado.

4. Arquitectura del Sistema

Anticithera es una Single Page Application (SPA) con un backend Jakarta EE. La aplicación funciona completamente offline una vez cargada; la conexión al backend solo se activa cuando el usuario inicia sesión para habilitar el respaldo en la nube.

4.1 Diagrama de Capas



Figura 1 — Capas de la arquitectura Anticithera

4.2 Backend: Estructura de Paquetes

```
com.anticithera.backend
├── config/
│   ├── CorsFilter.java      ← Filtro CORS con lista blanca de orígenes
│   ├── OptionsResource.java ← Responde preflight OPTIONS para todos los paths
│   └── RestApplication.java ← @ApplicationPath("/api")
├── entity/
│   ├── Usuario.java        ← id · username · email · password_hash
│   ├── SesionActividad.java ← token_sesion · inicio · fin · duracion
│   └── ExportacionZip.java  ← nombre_archivo · ruta_archivo · fecha
├── rest/
│   ├── AuthResource.java    ← POST /auth/register|login · POST /auth/logout
│   └── FileResource.java     ← POST /files/upload · GET /files/list|download/{id}
└── service/
    ├── AuthService.java     ← BCrypt hash · JWT UUID · sesión en BD
    └── FileService.java     ← Sanitización · escritura en disco · registro JPA
```

4.3 Esquema de Base de Datos

```
-- setup.sql
CREATE TABLE usuarios (
  id          SERIAL PRIMARY KEY,
  username    VARCHAR(50) UNIQUE NOT NULL,
  email       VARCHAR(100) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE sesiones_actividad (
  id          SERIAL PRIMARY KEY,
```

```
usuario_id      BIGINT UNSIGNED,  
token_sesion    VARCHAR(255) UNIQUE NOT NULL,  
inicio_conexion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
fin_conexion    TIMESTAMP,  
duracion_segundos INT,  
FOREIGN KEY (usuario_id) REFERENCES usuarios(id) ON DELETE CASCADE  
);  
  
CREATE TABLE exportaciones_zip (  
  id            SERIAL PRIMARY KEY,  
  usuario_id    BIGINT UNSIGNED,  
  nombre_archivo VARCHAR(255) NOT NULL,  
  ruta_archivo  VARCHAR(255) NOT NULL,  
  fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (usuario_id) REFERENCES usuarios(id) ON DELETE CASCADE  
);
```

4.4 Seguridad CORS

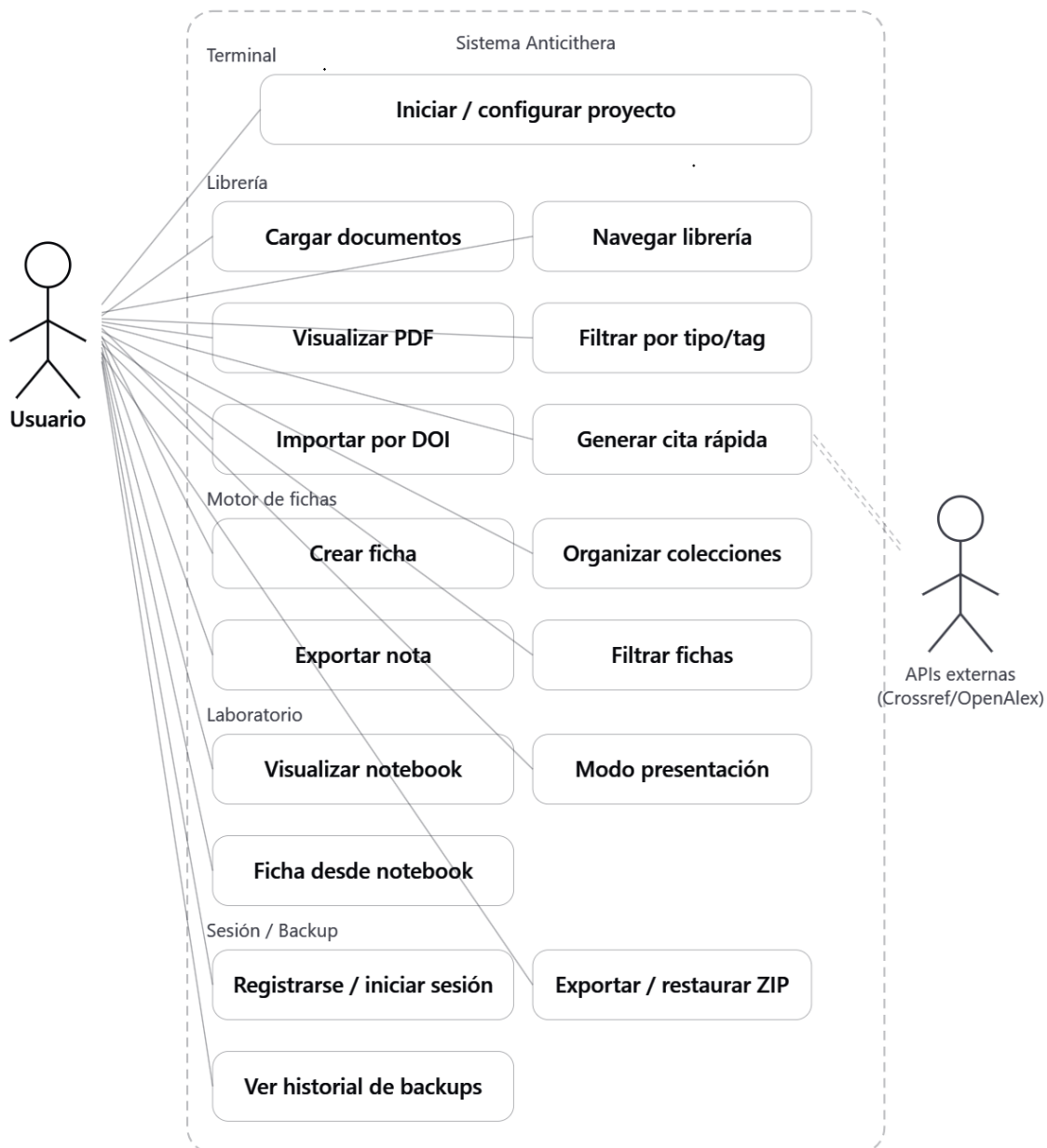
El filtro CorsFilter.java valida el encabezado Origin contra una lista blanca de dominios permitidos. Solo responde con cabeceras CORS si el origen coincide exactamente, evitando el acceso desde dominios no autorizados.

```
// CorsFilter.java – orígenes permitidos  
private static final List<String> ALLOWED_ORIGINS = Arrays.asList(  
  "http://127.0.0.1:5500",  
  "http://localhost:5500",  
  "https://crisstopher-ing-28.github.io"  
);
```

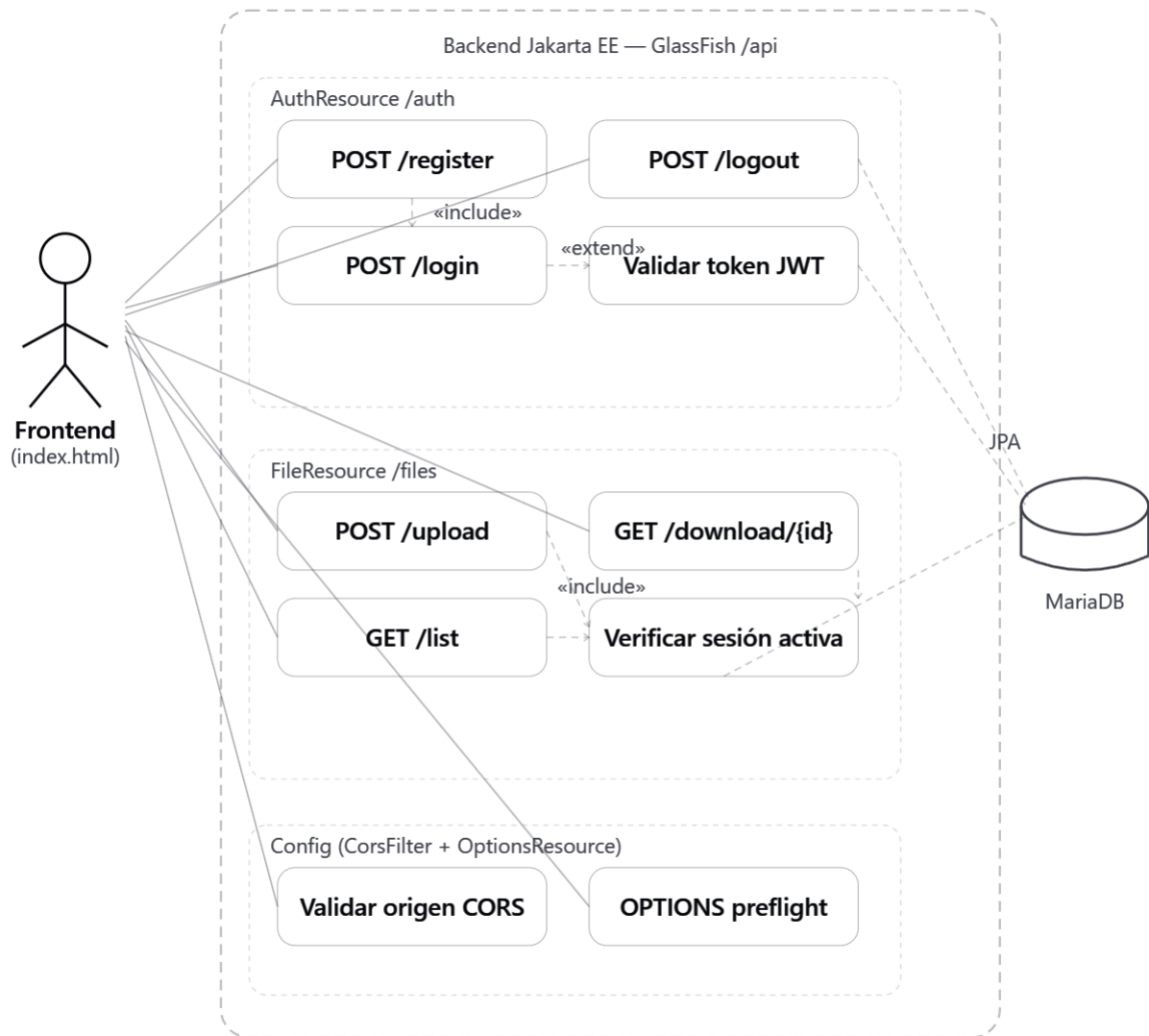

5. Diagramas de Casos de Uso

Los siguientes diagramas representan las interacciones entre los actores del sistema y las funcionalidades principales de Anticithera.

5.1 Actor: Usuario Investigador



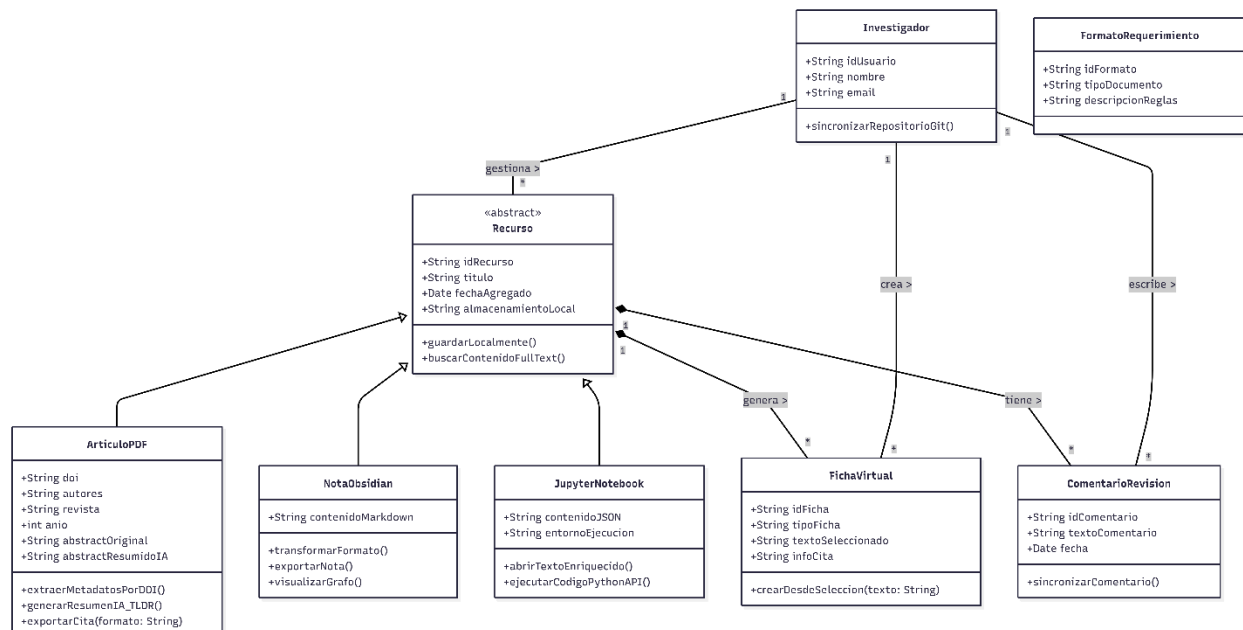
5.2 Sistema



5.3 Flujo de Trabajo Típico

Paso	Actor	Acción
1	Usuario	Selecciona tipo de documento (Tesis, Artículo...)
2	Dashboard	Inicializa entorno local en IndexedDB
3	Usuario	Sube PDF o vincula carpeta local
4	La Librería	Guarda referencia y metadatos en IndexedDB
5	Usuario	Selecciona fragmento de texto en el visor
6	Motor de Fichas	Abre modal con fragmento + referencia + página
7	Usuario	Añade comentario y almacena la ficha
8	Sistema	Persiste la ficha en IndexedDB store 'notes'
9	Usuario	Exporta librería como ZIP (+ sube si hay sesión)

5.4 Diagrama de clases



6. Internacionalización (i18n)

Anticithera soporta dos idiomas: Español e Inglés. Mediante un sistema de traducción basado en archivos JSON. Todos los textos visibles de la interfaz están externalizados en diccionarios de clave-valor que se cargan dinámicamente al cambiar el idioma.

6.1 Motor de i18n

```
// scripts.js – Motor de internacionalización
let currentLanguage = safeStorage.getItem('anticithera_lang') || 'es';
let translations = {};

async function loadLanguage(lang) {
  const response = await fetch(`./locales/${lang}.json`);
  translations = await response.json();
  currentLanguage = lang;
  safeStorage.setItem('anticithera_lang', lang);
  applyTranslations();
}

// Función helper para obtener un texto traducido
function t(key) {
  return translations[key] || key;
}
```

6.2 Atributos HTML para Traducción

Los elementos de la interfaz usan atributos data-* para vincularse automáticamente a las claves de traducción:

```
<!-- Texto de un botón -->
<button data-i18n="btn_create_project">Crear Proyecto</button>

<!-- Placeholder de un input -->
<input data-i18n-placeholder="input_library_name" />

<!-- Tooltip de un ícono -->
<i data-i18n-title="tooltip_export_library"></i>
```

6.3 Estructura de los Archivos JSON

Cada idioma corresponde a un archivo en el directorio ./locales/:

```
locales/
├── es.json  ← Diccionario en Español (idioma por defecto)
└── en.json  ← Diccionario en Inglés
```

```
// Fragmento representativo de es.json
{
  "btn_create_project": "Crear Proyecto",
  "toast_db_not_ready": "La base de datos no está lista aún.",
  "toast_prep_export": "Preparando exportación...",
  "toast_export_success": "¡Librería exportada con éxito!",
  "toast_restoring": "Restaurando contenido...",
  "prompt_restore_zip": "Pega una URL de ZIP o deja vacío para archivo local:",
```

```
"input_library_name": "Nombre de la Librería...",  
...  
}
```

```
// Fragmento representativo de en.json  
{  
  "btn_create_project": "Create Project",  
  "toast_db_not_ready": "Database is not ready yet.",  
  "toast_prep_export": "Preparing export...",  
  "toast_export_success": "Library exported successfully!",  
  "toast_restoring": "Restoring content...",  
  "prompt_restore_zip": "Paste a ZIP URL or leave blank for local file:",  
  "input_library_name": "Library Name...",  
  ...  
}
```

6.4 Cambio de Idioma en Tiempo Real

Al seleccionar un idioma en el selector (ES / EN) ubicado en la barra de navegación, la función `applyTranslations()` recorre todos los elementos con atributos `data-i18n`, `data-i18n-placeholder` y `data-i18n-title` actualizando su contenido sin recargar la página. La preferencia se persiste en `LocalStorage` y se restaura en la próxima visita.

7. Integración con Google Cloud (Google Drive)

Para permitir la importación de documentos desde Google Drive, se configuró un proyecto denominado «Anticithera» en Google Cloud Console. La integración utiliza OAuth 2.0 para la autenticación y la Google Picker API para que el usuario seleccione archivos desde su cuenta sin compartir credenciales.

7.1 Configuración en Google Cloud Console

1. Creado un proyecto en <https://console.cloud.google.com> con el nombre «Anticithera».
2. Habilitadas las APIs: Google Drive API y Google Picker API.
3. Crear credenciales OAuth 2.0 (tipo: Aplicación Web) y agregar los orígenes autorizados.
4. Obtener el Client ID y la API Key y agregarlos al frontend.

7.2 Implementación en el Frontend

```
// scripts.js – Constantes de configuración Google Cloud
Const CLIENT_ID =
const SCOPES = 'https://www.googleapis.com/auth/drive.readonly';

// Inicialización del cliente gapi
gapi.load('client:picker', () => {
  gapi.client.load('drive', 'v3');
  tokenClient = google.accounts.oauth2.initTokenClient({
    client_id: CLIENT_ID,
    scope: SCOPES,
    callback: handleAuthResponse
  });
});
```

7.3 Descarga de Archivos

Una vez que el usuario selecciona un archivo en el Picker, el sistema determina el tipo MIME y lo descarga usando la Drive API v3. Los documentos nativos de Google (Docs, Sheets, Slides) se exportan automáticamente a formatos compatibles (DOCX, XLSX, PDF).

```
// Lógica de descarga según tipo MIME
if (mimeType === 'application/vnd.google-apps.document') {
  url = `https://www.googleapis.com/drive/v3/files/${fileId}/export`
    + `?mimeType=application/vnd.openxmlformats-officedocument`
    + `.wordprocessingml.document`;
} else {
  url = `https://www.googleapis.com/drive/v3/files/${fileId}?alt=media`;
}
// Se descarga con el token OAuth en el header Authorization
```

8. Calidad y Seguridad del Código

8.1 SonarCloud — Análisis de Calidad

Se integró SonarCloud a través de la organización cristopher-ing-28 en github para analizar continuamente la calidad del código Java del backend. El último análisis arrojó los siguientes resultados:

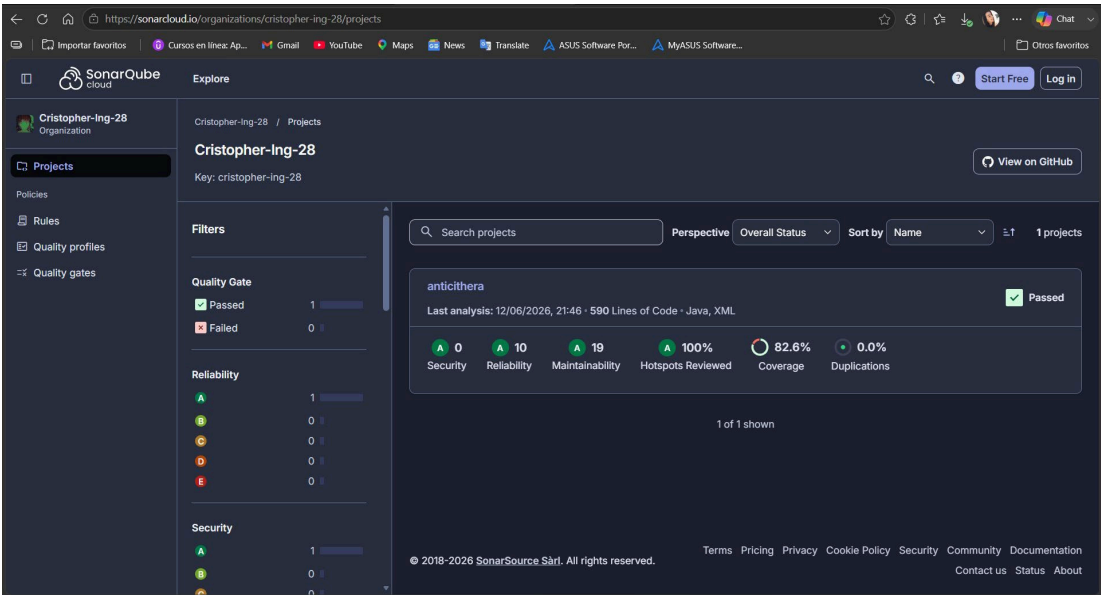


Figura 2 — Resultados de SonarCloud: Quality Gate Passed con 82.6% de cobertura

Desglose

Métrica	Calificación Valor	/	Detalle
Security	A (0 issues)		Sin vulnerabilidades de seguridad
Reliability	A (10 issues)		Issues menores de confiabilidad
Maintainability	A (19 issues)		Deuda técnica controlada
Hotspots Reviewed	100%		Todos los hotspots revisados
Coverage	82.6%		590 líneas de código Java analizado
Duplications	0.0%		Sin bloques de código duplicado
Quality Gate	PASSED ✓		Proyecto apto para producción

8.2 CodeQL — Análisis de Seguridad

CodeQL se configuró como parte del repositorio para analizar estáticamente el código en busca de vulnerabilidades conocidas. A diferencia de Checkmarx, CodeQL se ejecuta directamente en el pipeline de GitHub Actions sin costo adicional para proyectos académicos.

¿Por qué CodeQL en lugar de Checkmarx?

Checkmarx requiere licencia empresarial. CodeQL de GitHub es gratuito para proyectos públicos y privados bajo GitHub Advanced Security, con capacidades equivalentes para análisis de flujo de datos, inyección de SQL, XSS y otras vulnerabilidades OWASP.

```
# .github/workflows/codeql.yml (fragmento)
name: "CodeQL Advanced Security"
on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]
jobs:
  analyze:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: github/codeql-action/init@v3
        with:
          languages: java, javascript
      - uses: github/codeql-action/analyze@v3
```

8.3 Prácticas de Seguridad Implementadas

- Hashing de contraseñas con BCrypt (factor de costo configurable) — AuthService.java.
- Tokens de sesión UUID v4 (imposibles de predecir) — sin JWT firmado para simplificar, pero con expiración implícita por logout.
- Sanitización de nombres de archivo contra path traversal — FileService.sanitizeFileName().
- Validación de ruta canónica contra el directorio de uploads — previene directory traversal.
- CORS con lista blanca explícita de orígenes — CorsFilter.java.
- XSS mitigado con DOMPurify en el frontend para todo contenido HTML renderizado.

9. Referencia de la API REST

El backend Jakarta EE expone sus endpoints bajo la ruta base /api (definida en RestApplication.java con `@ApplicationPath("/api")`). Todos los endpoints consumen y producen JSON excepto /files/upload (multipart) y /files/download (octet-stream).

9.1 Autenticación — /api/auth

Método	Ruta	Body / Params	Respuesta
POST	/api/auth/register	{ username, email, password }	200: { token, username, email, startTime } 409: usuario/email duplicado
POST	/api/auth/login	{ username, password }	200: { token, username, email, startTime } 401: credenciales inválidas
POST	/api/auth/logout	Header: Authorization: Bearer {token}	200 OK 400: sin token

```
// Ejemplo de registro – fetch desde el frontend
const res = await fetch(`${API_BASE_URL}/auth/register`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ username: 'investigador01',
    email: 'inv@univ.mx',
    password: 'S3cret!' })
});
const { token, username } = await res.json();
```

9.2 Archivos ZIP — /api/files

Método	Ruta	Params	Respuesta
POST	/api/files/upload	Header Auth + Query fileName + Body: blob ZIP	200: ExportacionZip JSON 401 500
GET	/api/files/list	Header: Authorization: Bearer {token}	200: [{ id, nombreArchivo, fechaCreacion }]
GET	/api/files/download/{id}	Header Auth + Path param id	200: binary ZIP 401 404

```
// Subida automática del ZIP al exportar (exportZIP.js)
const response = await fetch(
  `${API_BASE_URL}/files/upload?fileName=${encodeURIComponent(nombreZip)}`,
  {
    method: 'POST',
    headers: { 'Authorization': `Bearer ${userSession.token}` },
    body: contenidoZip // Blob del archivo ZIP
  }
);
```

10. Guía de Despliegue

10.1 Requisitos del Sistema

Componente	Requisito
JDK	Java Development Kit 17 o superior
Servidor de aplicaciones	GlassFish 7 (Jakarta EE 10)
Base de datos	MariaDB 10.6+ o MySQL 8+
Frontend	Cualquier servidor estático (GitHub Pages, Apache, Nginx)
Navegador	Chrome 90+, Firefox 88+, Edge 90+ (con soporte IndexedDB)

10.2 Configuración del Backend

1. Instalar JDK 17 y GlassFish 7.
2. Crear la base de datos ejecutando setup.sql:

```
mysql -u root -p < setup.sql
```

3. Configurar la conexión JPA en persistence.xml apuntando a MariaDB.
4. Añadir los .jar de dependencias al dominio GlassFish:

```
# Dependencias requeridas en glassfish/domains/domain1/lib/
jjwt-api-0.11.5.jar
jjwt-impl-0.11.5.jar
jjwt-jackson-0.11.5.jar
jbcrypt-0.4.jar
mariadb-java-client-3.x.jar
```

5. Desplegar el archivo .war en GlassFish:

```
# Panel de administración: https://localhost:4848
# O mediante asadmin:
asadmin deploy anticithera-backend.war
```

6. Verificar que el backend responde en https://localhost:8181/api/auth/login.

10.3 Configuración del Frontend

- Clonar el repositorio o descargar los archivos del frontend.
- Actualizar la constante API_BASE_URL en scripts.js con la URL del backend:

```
// scripts.js – línea 8 (aprox.)
const API_BASE_URL = 'https://tu-servidor:8181/api';
```

Crear los archivos de localización si no existen:

```
locales/
├── es.json
└── en.json
```

- Publicar en GitHub Pages o cualquier servidor estático. El frontend funciona completamente sin el backend para funciones básicas de servicio.

11. Glosario

Librería / Herramienta	Descripción y URL
Tailwind CSS	Framework CSS utilitario — https://tailwindcss.com
Lucide Icons	Iconos ligeros y personalizables — https://lucide.dev
Marked.js	Parser de Markdown de alto rendimiento — https://marked.js.org
DOMPurify	Sanitizador HTML contra XSS — https://github.com/cure53/DOMPurify
Notebook.js	Renderizador de Jupyter Notebooks en el navegador — https://github.com/jsvine/notebookjs
MathJax	Motor de visualización de ecuaciones LaTeX — https://www.mathjax.org
PDF.js	Visor de PDF en el navegador — https://mozilla.github.io/pdf.js
FileSaver.js	Guardar archivos desde el cliente — https://github.com/eligrey/FileSaver.js
JSZip	Crear y leer archivos ZIP en JS — https://stuk.github.io/jszip
Prism.js	Resaltado de sintaxis — https://prismjs.com
Mammoth.js	Convertir .docx a HTML — https://github.com/mwilliamson/mammoth.js
Google APIs JS	Integración con servicios Google — https://developers.google.com/identity
Mermaid.js	Diagramas desde texto — https://mermaid.js.org
Jakarta EE 10	Especificación empresarial Java — https://jakarta.ee
GlassFish 7	Servidor de aplicaciones Jakarta EE — https://glassfish.org
jjwt 0.11.5	Librería JWT para Java — https://github.com/jwt/jjwt
jBCrypt	Implementación BCrypt para Java — https://www.mindrot.org/projects/jBCrypt
SonarCloud	Análisis de calidad continuo — https://sonarcloud.io
CodeQL	Análisis estático de seguridad — https://codeql.github.com
Jenkins	Servidor de integración continua — https://www.jenkins.io